

Research Dataset Catalogue & Bibliographic Metadata Manager

DS3294: DS Practice — Project #14 — March 2026

1. Project Objective

The goal of this project is to design and implement a reproducible, well-documented bibliographic metadata management system for research datasets. The system collects 100 bibliographic records from open-access repositories, validates and normalises the metadata, stores it in a structured database, and provides an efficient query interface for searching by author, year, keyword, DOI, and repository.

Key capabilities the system must demonstrate:

- Automated ingestion from REST APIs and structured file formats (JSON, CSV, BibTeX)
- Validation, normalisation, and deduplication of bibliographic metadata
- Relational and/or document-based persistent storage
- Query interface with performance benchmarking across indexing strategies
- Versioning and incremental update support for stored records

2. Steps Taken

- **Source Identification:** Identified four open-access data sources: Zenodo, Kaggle, OpenAlex, and DataCite.
- **Ingestion Scripts:** Wrote `api_fetcher.py` with four fetch functions — one per source — handling pagination and rate limits automatically.
- **File Parser:** Wrote `file_parser.py` supporting BibTeX, JSON, and CSV formats with auto-detection from file extension.
- **Pipeline:** Wrote `ingest_pipeline.py` to orchestrate all fetches, apply MD5-based deduplication, tag each record with its source, and save output to `data/raw/all_raw.json`.
- **Testing:** Wrote `test_parser.py` to demonstrate the parser on `iris.csv` and `sample.json` files.
- **Database Schema:** Designed a relational database schema using SQLite, with pandas and SQL queries, to efficiently store and manage bibliographic metadata.
- **Version Control:** Initialised GitHub repository with README, `requirements.txt`, and ingestion module.

3. Major Algorithms, Processes & Math

3.1 MD5 Hashing for Deduplication

Each collected record is serialised to a canonical JSON string with sorted keys and passed through the MD5 hash function to produce a 128-bit (32-character hexadecimal) fingerprint:

```
hash = MD5( JSON.stringify(record, sort_keys=True) )
```

If two records produce the same hash they are identical regardless of source or filename. Only the first occurrence is retained. This runs in $O(n)$ time over n records.

3.2 Pagination & Rate-Limit Handling

Each API fetcher loops over pages until the target record count is reached. On receiving HTTP 429 (Too Many Requests), the fetcher sleeps for 10 seconds and retries — ensuring compliance with server-side rate limits without manual intervention.

3.3 Fuzzy Matching for Duplicate Detection

Duplicate detection is performed using the RapidFuzz library. It computes a similarity score between two dataset titles based on edit distance.

```
similarity = fuzz.ratio(title1, title2)
```

The similarity score ranges from 0 to 100. If the score exceeds a threshold (e.g., 90%), the records are considered potential duplicates.

3.4 Edit Distance (Levenshtein Distance)

Edit distance measures the minimum number of operations (insert, delete, replace) required to transform one string into another. A smaller distance implies higher similarity.

3.5 Database Indexing

Indexes were created on frequently queried fields such as authors, title, and year. Indexing improves query performance by reducing search complexity from linear time to logarithmic time.

4. Current Status of Work

Component	Status	Owner
GitHub Repository & README	Complete	Team
Data Collection, File Parser (CSV, JSON, BibTeX)	In progress	Abhinav
MD5 Deduplication Pipeline	In progress	Abhinav
100 Raw Records Collected	Complete	Abhinav
Database Schema Design	In Progress	Gourav
Validation & Normalisation	In Progress	Siyag
Query Interface & Benchmarking	In Progress	Nikunj

5. Team Work Distribution

Member	Role	Responsibilities
Abhinav Debbarma	Data Collection & Ingestion	API fetchers, file parser, MD5 deduplication, 100 raw records
Gourav Kumawat	Database Design & Schema	Schema design (SQLite), duplicate detection, record merging
Parulekar Siyag Avinash	Validation, Normalisation & Versioning	Field validation, normalisation, conflict resolution, versioning, dashboard
Nikunj	Query Interface & Performance	Search interface, indexing strategy comparison, benchmarking, dashboard

6. Expected Output

The final deliverable is a reproducible, end-to-end bibliographic metadata management system containing 100 or more curated research dataset records. The system will demonstrate:

- A fully automated ingestion pipeline fetching from multiple open-access sources with zero manual intervention
- A clean, validated, deduplicated dataset stored in relational database SQLite
- A query interface supporting search by author, year, keyword, DOI, and repository with documented response times
- A performance comparison between indexed and non-indexed fields across both storage back-ends
- Complete version history for all records with soft-delete support
- Full codebase on GitHub with README, requirements, and documented pipeline logs
- A full interactive dashboard with various tasks like search options, analysis, author information, etc.